

Тестове завдання

№1

SQL

Дана таблиця співробітників.

	id	name	department	salary
0	11	Дарія	hr	70
1	12	Борис	hr	78
2	21	Олена	it	84
3	22	Ксенія	it	90
4	24	Марина	it	104
5	23	Леонид	it	104
6	25	Іван	it	120
7	31	Вероника	sales	96
8	32	Григорій	sales	96
9	33	Ганна	sales	100

	id	name	department	salary	rank	revenue_cumsum
0	25	Іван	it	120	10	120
1	23	Леонид	it	104	9	224
2	24	Марина	it	104	8	328
3	33	Ганна	sales	100	7	428
4	32	Григорій	sales	96	6	524
5	31	Вероника	sales	96	5	620
6	22	Ксенія	it	90	4	710
7	21	Олена	it	84	3	794
8	12	Борис	hr	78	2	872
9	11	Дарія	hr	70	1	942

1) Спершу я створив спільний табличний вираз `table_rank_salary`, у якому вибрав усі необхідні поля та застосував функцію `ROW_NUMBER() OVER(ORDER BY salary ASC)`, щоб пронумерувати працівників від найменшої заробітної плати до найбільшої. Завдяки цьому співробітник із найнижчою зарплатою отримав ранг 1, а з найвищою — ранг 10. Далі в основному запиті я витягнув ці дані та розрахував кумулятивну суму за допомогою віконної функції `SUM(salary) OVER(ORDER BY rank_person_salary DESC)`, яка рахує наростаючий підсумок зарплат у зворотному напрямку — починаючи від найвищого рангу до найнижчого, а в кінці відсортуював підсумковий результат за спаданням рангу.

2) Червоними колами на слайді підкреслено різницю в порядку рядків між очікуваною таблицею зліва та фактичним виводом у `pgAdmin` справа. Вона виникла для співробітників із однаковими заробітними платами — Леоніда й Марини (по 104), а також Григорія та Вероніки (по 96). Оскільки в сортуванні `ORDER BY salary ASC` не було задано додаткового критерію (наприклад, за `id`), функція `ROW_NUMBER()` розподілила порядкові номери недетерміновано, через що в лівій таблиці раніше йде Леонід, а в реальному виводі бази даних вище опинилася Марина. Щоб порядок завжди був стабільним і збігався, у віконну функцію достатньо додати вторинне поле сортування, наприклад `ORDER BY salary ASC, id ASC`.

```
1 -- Task 1:
2 -- Скласти рейтинг працівників за розміром заробітної плати.
3 -- Розрахувати кумулятивну суму за цим
4 WITH table_rank_salary AS(
5     SELECT
6         id, name, department, salary
7         , ROW_NUMBER() OVER(ORDER BY salary ASC) AS rank_person_salary
8     FROM employees
9 )
10 SELECT
11     id, name, department, salary, rank_person_salary
12     , SUM(salary) OVER(ORDER BY rank_person_salary DESC) AS cum_sum
13 FROM table_rank_salary
14 ORDER BY rank_person_salary DESC;
```

Data Output

	id	name	department	salary	rank_person_salary	cum_sum
1	25	Іван	it	120	10	120
2	24	Марина	it	104	9	224
3	23	Леонид	it	104	8	328
4	33	Ганна	sales	100	7	428
5	32	Григорій	sales	96	6	524
6	31	Вероника	sales	96	5	620
7	22	Ксенія	it	90	4	710
8	21	Олена	it	84	3	794
9	12	Борис	hr	78	2	872
10	11	Дарія	hr	70	1	942

Я виніс `ROW_NUMBER()` у CTE, тому що SQL не дозволяє посилатися на новостворений аліас або вкладати віконні функції одна в одну в межах одного `SELECT`.

Крім того, якби я спробував порахувати кумулятивну суму одразу через `SUM(salary) OVER(ORDER BY salary DESC)` без унікального рангу, то для однакових зарплат (по 104 і 96) база за замовчуванням додала б суму разом усім дублікатам (вийшло б 328 і 328 замість покрокових 224 і 328). CTE дозволив мені спочатку створити чіткий унікальний порядок рядків, а вже потім коректно нараховувати суму крок за кроком.

	id	name	department	salary
0	11	Дарія	hr	70
1	12	Борис	hr	78
2	21	Олена	it	84
3	22	Ксенія	it	90
4	24	Марина	it	104
5	23	Леонид	it	104
6	25	Іван	it	120
7	31	Вероника	sales	96
8	32	Григорій	sales	96
9	33	Ганна	sales	100

	id	name	department	salary	row_number
0	12	Борис	hr	78	1
1	11	Дарія	hr	70	2
2	25	Іван	it	120	1
3	24	Марина	it	104	2
4	23	Леонид	it	104	3
5	22	Ксенія	it	90	4
6	21	Олена	it	84	5
7	33	Ганна	sales	100	1
8	31	Вероника	sales	96	2
9	32	Григорій	sales	96	3

```

17 -- Task 2:
18 -- Той самий рейтинг, тільки не для всієї компанії, а по кожному департаменту
19 SELECT
20     id, name, department, salary
21     , ROW_NUMBER() OVER(PARTITION BY department ORDER BY salary DESC) AS row_number_
22 FROM employees
23 ORDER BY department, row_number_;
24

```

Data Output

	id [PK] integer ↗	name character varying (50) ↗	department character varying (50) ↗	salary integer ↗	row_number_bigint 🔒
1	12	Борис	hr	78	1
2	11	Дарія	hr	70	2
3	25	Іван	it	120	1
4	23	Леонид	it	104	2
5	24	Марина	it	104	3
6	22	Ксенія	it	90	4
7	21	Олена	it	84	5
8	33	Ганна	sales	100	1
9	31	Вероника	sales	96	2
10	32	Григорій	sales	96	3

1) Для цього я використав віконну функцію `ROW_NUMBER()` `OVER(PARTITION BY department ORDER BY salary DESC)`. Конструкція `PARTITION BY department` розділяє всю таблицю на окремі ізольовані групи (вікна) за відділами — `hr`, `it` та `sales`, а `ORDER BY salary DESC` всередині кожного департаменту сортує працівників від найвищої оплати до найнижчої. Завдяки цьому нумерація рядків автоматично скидається на 1 на початку кожного нового відділу: у відділі `hr` першим іде Борис (78), у відділі `it` лідирує Іван (120), а в `sales` — Ганна (100). У фінальному блоці `ORDER BY department, row_number` я згрупував результат на екрані, щоб співробітники виводилися послідовно за назвою відділу та місцем у рейтингу.

2) Червоними кружечками на слайді виділено пари співробітників з однаковими заробітними платами: Марину та Леоніда (по 104 у відділі `it`), а також Вероніку та Григорія (по 96 у відділі `sales`). У референсній таблиці зліва ранг 2 в IT-відділі дістався Марині, а ранг 3 — Леониду, тоді як у фактичному результаті pgAdmin справа другим став Леонид, а третьою — Марина. Причина в тому, що у виразі `ROW_NUMBER() OVER(PARTITION BY department ORDER BY salary DESC)` сортування задане лише за полем `salary`. Якщо значення заробітної плати повністю збігаються, база даних розставляє порядкові номери недетерміновано, оскільки не має додаткового критерію, кого ставити вище. Щоб порядок рядків був стабільним і повністю передбачуваним, у сортування вікна слід додати тай-брейкер за унікальним полем, наприклад: `ORDER BY salary DESC, id ASC`.

№3

SQL

	id	name	department	salary		id	name	department	salary	diff_from_prev	
0	11	Дарія	hr	70		0	11	Дарія	hr	70	NaN
1	12	Борис	hr	78		1	12	Борис	hr	78	11.0
2	21	Олена	it	84		2	21	Олена	it	84	7.0
3	22	Ксенія	it	90		3	22	Ксенія	it	90	7.0
4	24	Марина	it	104		4	31	Вероника	sales	96	6.0
5	23	Леонид	it	104		5	32	Григорій	sales	96	0.0
6	25	Іван	it	120		6	33	Ганна	sales	100	4.0
7	31	Вероника	sales	96		7	24	Марина	it	104	4.0
8	32	Григорій	sales	96		8	23	Леонид	it	104	0.0
9	33	Ганна	sales	100		9	25	Іван	it	120	15.0

25

-- Task 3:

26

-- Упорядкувати працівників щодо зростання зарплати та перевірити,

27

-- чи великий розрив між сусідами у відсотках

28

SELECT

29

id, name, department, salary

30

, ROUND((salary - LAG(salary) OVER(ORDER BY salary))*100.0

31

/ LAG(salary) OVER(ORDER BY salary),1) AS diff_from_prev

32

FROM employees

33

ORDER BY salary;

Data Output

≡

📄

📄

🗑

📦

📄

📄

SQL

	id	name	department	salary	diff_from_prev
	[PK] integer	character varying (50)	character varying (50)	integer	numeric
1	11	Дарія	hr	70	[null]
2	12	Борис	hr	78	11.4
3	21	Олена	it	84	7.7
4	22	Ксенія	it	90	7.1
5	31	Вероника	sales	96	6.7
6	32	Григорій	sales	96	0.0
7	33	Ганна	sales	100	4.2
8	23	Леонид	it	104	4.0
9	24	Марина	it	104	0.0
10	25	Іван	it	120	15.4

Для цього я використав віконну функцію `LAG(salary) OVER(ORDER BY salary)`, яка повертає значення заробітної плати попереднього рядка у впорядкованому списку. Щоб отримати відносну різницю у відсотках, я обчислив абсолютний приріст `(salary - LAG(salary) OVER(...))`, помножив його на `100.0` для переведення у відсотки та дробовий формат, поділив на базову зарплату попередника `LAG(salary) OVER(...)` і округлив результат функцією `ROUND(..., 1)` до одного знака після коми.

Для першого рядка (Дарія, 70) значення попередника відсутнє, тому функція повертає `[null]` (або `NaN` у референсній таблиці), що є абсолютно коректним. Для працівників з однаковою зарплатою (Григорій після Вероніки, Марина після Леоніда) різниця дорівнює `0.0`. Наприкінці весь набір даних відсортовано через `ORDER BY salary`, щоб вивести співробітників у чіткій послідовності від найменшої оплати до найбільшої.

Якщо ми хочемо просто обрізати десяті, соті, то:

```
TRUNC((salary - LAG(salary) OVER(ORDER BY salary)) * 100.0 / LAG(salary) OVER(ORDER BY salary), 0)
```

№4 SQL

4. Порівняння з фондом оплати праці: Кожен департамент має фонд оплати праці — грошову суму, яка щомісяця йде на виплату зарплат співробітникам. Подивитися, який відсоток від цього фонду становить зарплата кожного працівника

	id	name	department	salary		id	name	department	salary	fund_share	
0	11	Дарія	hr	70	→	0	11	Дарія	hr	70	47.3
1	12	Борис	hr	78		1	12	Борис	hr	78	52.7
2	21	Олена	it	84		2	21	Олена	it	84	16.7
3	22	Ксенія	it	90		3	22	Ксенія	it	90	17.9
4	24	Марина	it	104		4	24	Марина	it	104	20.7
5	23	Леонид	it	104		5	23	Леонид	it	104	20.7
6	25	Іван	it	120		6	25	Іван	it	120	23.9
7	31	Вероника	sales	96		7	31	Вероника	sales	96	32.9
8	32	Григорій	sales	96		8	32	Григорій	sales	96	32.9
9	33	Ганна	sales	100		9	33	Ганна	sales	100	34.2

```
35 -- Task 4:
36 SELECT
37     id, name, department, salary
38     , ROUND(salary*100.0 / (SUM(salary) OVER(PARTITION BY department)),1) AS fund_share
39 FROM employees
40 ORDER BY department;
```

Data Output

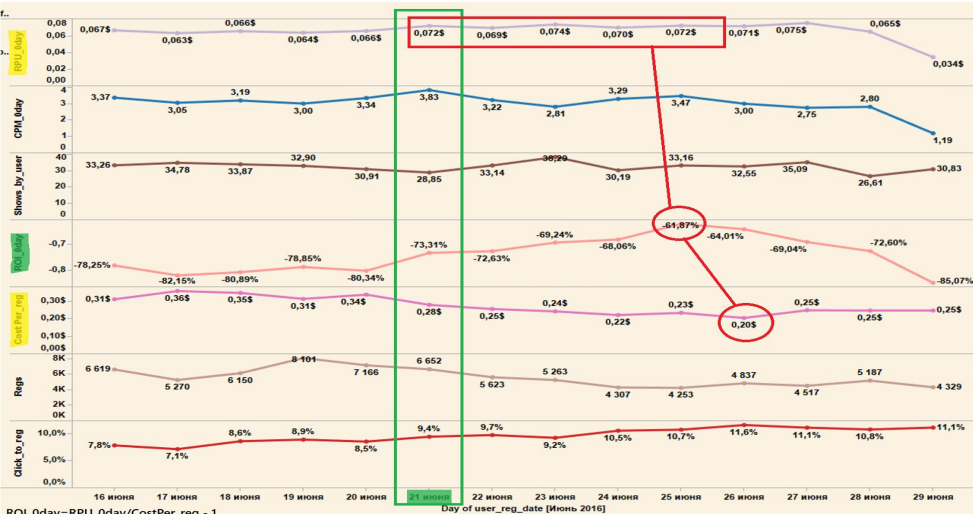
	id	name	department	salary	fund_share
	[PK] integer	character varying (50)	character varying (50)	integer	numeric
1	11	Дарія	hr	70	47.3
2	12	Борис	hr	78	52.7
3	21	Олена	it	84	16.7
4	22	Ксенія	it	90	17.9
5	24	Марина	it	104	20.7
6	23	Леонид	it	104	20.7
7	25	Іван	it	120	23.9
8	31	Вероника	sales	96	32.9
9	32	Григорій	sales	96	32.9
10	33	Ганна	sales	100	34.2

Для цього я використав віконну агрегатну функцію `SUM(salary) OVER(PARTITION BY department)`. Конструкція `PARTITION BY department` розбиває обчислення на окремі групи за відділами й рахує тотальну суму зарплат конкретно для `hr`, `it` та `sales`, не згортаючи при цьому рядки таблиці, як це робить звичайний `GROUP BY`.

Щоб визначити частку, я помножив зарплату працівника на `100.0` для переведення у відсотки й отримання дробового типу даних, поділив на розрахований сумарний фонд департаменту та за допомогою функції `ROUND(..., 1)` округлив значення до одного десяткового знака. У кінці запиту застосував `ORDER BY department`, щоб згрупувати співробітників за їхніми підрозділами.

Завдання 2.1 зроблено в jupyter.ipynb

Завдання 2.2



RPU_Day Средний доход на користувача в нульовий день його життя на проєкті (у день реєстрації)
CPM_Day Середня вартість 1000 показів реклами для користувача в нульовий день
Shows_by_user Середня кількість показів на користувача
ROI_Day інвестиції розрахований у нульовий день тільки за доходом у цей день
CostPer_reg Середня вартість реєстрації
Regs Кількість реєстрацій
Click_to_reg Відсоток користувачів, які зареєструвалися до користувачів, які клікнули по нашому рекламному банеру
Нульовий день – день реєстрації

ROI_Day за своєю природою $\approx \text{RPU_Day} / \text{CostPer_reg} - 1$, тому його динаміка повністю пояснюється співвідношенням цих двох метрик

Період з 21.06

Динаміка ROI_Day: Показник стабільно покращується з -73,31% на 21.06 до свого піку -61,87% на 25.06. Після 25 числа починається поступовий спад: 26.06 ROI знижується до -64,01%, а 27.06 — до -69,04%.

Драйвери змін: Покращення окупності відбулося насамперед через здешевлення реєстрацій: CostPer_reg поступово впав з \$0,28 до \$0,20 (досягнувши мінімуму 26.06). При цьому дохідність користувачів (RPU_Day) у цей проміжок залишалася стабільною на рівні $\approx \$0,069$ – $0,075$.

Ключові взаємозв'язки: Відсоток реєстрацій (Click_to_reg) стабільно зростає, досягаючи 11,6% на 26.06, тоді як загальний обсяг реєстрацій (Regs) знижується порівняно з піком 19.06. Це свідчить про зуження аудиторії за рахунок більш цільового, але менш об'ємного трафіку.

Можливі причини: Такий тренд зазвичай є наслідком успішної оптимізації ставок/таргетингу, переходу на дешевші та більш релевантні джерела трафіку, або ж тимчасового зниження конкуренції на аукціоні.

Подальші дії: Необхідно з'ясувати, які саме налаштування кампаній діяли до 25.06, щоб оцінити можливість повернення до цього підходу. Також варто моніторити Click_to_reg разом із загальною кількістю реєстрацій, щоб не допустити втрати сукупного доходу через занадто сильне урізання обсягів трафіку.

Період з 28.06

Динаміка ROI_Day: Відбувається різке погіршення: показник падає до -72,60% на 28.06 і досягає найгіршого значення за весь місяць — -85,07% на 29.06.

Драйвери змін: Збіг двох негативних факторів: вартість реєстрації (CostPer_reg) зросла та зафіксувалася на рівні \$0,25, тоді як дохідність (RPU_Day) обвалилася майже вдвічі — з \$0,065 (28.06) до \$0,034 (29.06).

Ключові взаємозв'язки: Разом із різким падінням RPU_Day 29.06 спостерігається таке ж різке падіння вартості показів (CPM_Day) до \$1,19. Одночасне падіння цих двох метрик в один день є вкрай нетиповим.

Можливі причини: Погіршення, що почалося напередодні та загострилося 28-29 числа, свідчить про залучення дешевого, неплатоспроможного трафіку або вигорання креативів. Аномалія 29.06 дуже схожа на масштабний технічний збій у системі трекінгу чи атрибуції доходу.

Подальші дії: Терміново розбити RPU_Day та CostPer_reg за каналами та географією для локалізації сегмента, що спричинив обвал 29.06. Перевірити коректність роботи трекінгу за цей день. У майбутньому — проводити контрольовані A/B-тести для нових джерел трафіку перед перерозподілом бюджетів.

Завдання 2.3 зроблено в jupyter.ipynb

Завдання 2.4 зроблено в jupyter.ipynb